

Darmstadt University of Applied Sciences

– Faculty of Computer Science –

AI driven evolution of penetration testing in software security

by

Henning Prien

Matriculation number: 1132644

First Examiner : Dominik Machold

DECLARATION

I hereby certify that I have written the present paper independently and have not used any sources other than those listed in the bibliography.

All passages that have been taken literally or in substance from published or unpublished sources are identified as such.

The drawings or illustrations in this work were created by myself or are provided with an appropriate source reference.

This work has not been submitted to any other examination authority in the same or a similar form.

Darmstadt, March 13, 2026

Henning Prien

ABSTRACT

Software forms the backbone of modern society, supporting critical infrastructure such as communication networks, financial systems, and healthcare services. The security of these systems is a pressing concern, as vulnerabilities can lead to severe consequences, including financial losses, reputational harm, and even threats to human safety. While software vendors are motivated by economic incentives and legal obligations—such as compliance with regulations like the EU's General Data Protection Regulation (GDPR)—to prioritize security, the rapidly evolving threat landscape presents significant challenges. Emerging threats like advanced persistent attacks, ransomware, and supply chain vulnerabilities require more dynamic and scalable security solutions than traditional methods can offer.

This thesis investigates the potential of Artificial Intelligence (AI)-driven penetration testing, specifically through the use of Large Language Model (LLM)s, to automate and improve the process of identifying software vulnerabilities. The study begins by situating penetration testing within the Software Development Livecycle (SDLC), highlighting its role as a creative and adaptive method for simulating real-world cyberattacks. It then explores the transition from manual testing to AI-assisted approaches, analyzing recent developments such as PentestGPT, PentestAgent, and ARTEMIS. These advancements demonstrate that LLMs, when combined with domain-specific knowledge, can effectively perform penetration tests in controlled settings.

However, real-world applications reveal certain limitations. Tools like XBOW, despite their success in bug bounty programs, often generate high rates of false positives and struggle with proprietary or niche technology stacks. Security experts note that while AI-driven tools excel at identifying known vulnerabilities, they lack the lateral thinking and contextual adaptability that human testers provide. Additionally, security risks such as data leakage through prompt injection and biases in training data pose further challenges.

The thesis concludes that AI can substantially accelerate penetration testing, but it is not yet a substitution for human expertise. Future research should focus on enhancing AI's ability to navigate proprietary systems, reducing false positives, and addressing security risks. By overcoming these challenges, AI-driven penetration testing can become a dependable and scalable solution, complementing and ultimately strengthening security efforts.

CONTENTS

I Thesis
1 Introduction 2
1.1 Problem Statement . . . 2
1.2 Goal of research . . . 2
1.3 Research Question and Hypotheses . . . 3
1.4 Structure . . . 3
1.5 Methodology . . . 3
2 State of Research and related Work 4
3 Measures of Software Security 5
4 Penetration Testing 7
5 AI driven Penetration Testing 9
6 Conclusion and Outlook 12
6.1 This is possible . . . 12
6.2 Functional limitations . . . 12
6.3 Security Challenges . . . 13
6.4 Future Work . . . 13

II Appendix

Bibliography 15

ABBREVIATIONS

SDLC Software Development Livecycle

SAST Static Application Security Test

DAST Dynamic Application Security Test

CIA Confidentiality Indtegrity Availability

AI Artificial Intelligence

LLM Large Language Model

CTF Capture The Flag

CLI Command Line Interface

RAG Retrieval-augmented generation

GDPR General Data Protection Regulation

Part I

THESIS

1 INTRODUCTION

Wide ranges of day-to-day life rely on software. Communication technologies, payment services as well as health systems all rely on software. Failure, unavailability or data leaks in such crucial services may lead to severe harm or even death to human. The society has a great indirect interest in software security as it heavily relies on digital infrastructure. [1]

Not only the society but also software vendors have multiple interests in providing secure software. On the one hand, companies financially benefit from the voluntary disclosure of security measures they took. For example their market value is influenced in a positive way by this disclosure. This is due to more transparency and proactive actions. [2]

On the other hand, software vendors must comply with several laws and regulations regarding data protection [3]. Penalties in form of "fines up to 20 000 000 EUR, or in the case of an undertaking, up to 4 % of the total worldwide annual turnover" [3] threaten to break these laws.

1.1 PROBLEM STATEMENT

These needs of cyber security need to be satisfied while surrounded by a dynamically and fast evolving threat landscape. State and non-state actors drive a wide and diverse landscape of cyber security threats. "Supply chain attacks, advanced persistent threats, ransomware, and vulnerabilities on the Internet of Things" [4] are just some cyber security threats modern systems face. Thus developing and maintaining software systems securely and protecting them from any harm through cyber criminals gets harder ever since.

1.2 GOAL OF RESEARCH

To address these growing difficulties of securing software systems, corporate vendors and developers can apply several security measures.

The SDLC describes multiple steps in which the development of secure software can be split up. One central part of the development of secure software is testing. [5] A highly effective approach of software testing is Penetration Testing [6]. Penetration Testing used to be a manual and creative approach of dynamically testing for security vulnerabilities in software. However, traditional thus manual methods of Penetration Testing are no longer sufficient to keep track of fast evolving threat landscapes and ever growing software systems. This raises the need of automating and speeding up Penetration Testing. [7]

This term paper will discuss how AI-driven systems can support penetration testers. It will also discuss to which degree Penetration Testing already

can be automate and thus help Penetration Testers to focus of more advances research and testing.

1.3 RESEARCH QUESTION AND HYPOTHESES

The central research question addressed in this term paper is following:

How do AI-powered tools improve and automate the process of software penetration testing and which risks come with it?

The Hypotheses to answer this question is that LLMs are capable of performing Penetration Tests when they are provided with sufficient resources. These systems are not capable of performing end-to-end penetration testing without a human in the loop. Hallucination of LLMs lead to false positives.

1.4 STRUCTURE

First of all, a foundation will be built up to give the content of this term paper a context. This will be done my the definition of important and related terms. Following, related historic approaches as well as related work will be presented in the context of the research question.

The main content of this term paper will consist of tree parts. To emphasize the role of Penetration Testing in secure software development, the first step will be to differentiate between measures of software security. Second step to answer the research question will be the introduction to Penetration Testing. Followed by third, taking a look at AI-driven Penetration Testing.

In the conclusion all information will be summarized and related to each other. In this part also the research question will be answered based on the findings.

1.5 METHODOLOGY

A literature review will discuss past and ongoing research on the automation of Penetration Testing. Capabilities of earlier solutions will be compared to the current state of research.

The usage of LLMs for automated Penetration Testing currently evolves fast. This is why special attention will be drawn to security experts' reports of experience. As these experts actually working with these tools and thus know their capabilities. This will ensure to cover the most recent evolutions in the field of automated Penetration Testing.

Based on the literature review, attention will also be drawn to security challenges and the need of future work.

2 STATE OF RESEARCH AND RELATED WORK

This term paper is partially based on research papers that are still in preprint or have not received much attention yet because they have been published recently. As the technology of AI and especially LLMs is still evolving and heavily underlies broad attention, some aspects might change and evolve in future.

There are three main differences regarding the knowledge based on which the penetration test is performed. Black-, Gray- and White-Box penetration testing. [8]

Traditional penetration testing tooling already implemented various automation to support Penetration Testers [9].

Earlier approaches to automate the process of Penetration Testing leveraged Reenforcement Learning [10]. However scaling these approaches to an production ready state brought up difficulties [11].

Leveraging LLMs for penetration testing shows better results and scalability. The project PentestGPT also describes an agentic approach for orchestrating LLMs beneficially. [12]

Multiple tools already implement AI for penetration testing to support human penetration testers [13].

3 MEASURES OF SOFTWARE SECURITY

Multiple models and frameworks suggest steps that can be taken to develop secure software. They all describe different planning and implementation phases to cover every step of software engineering. [14]

The SDLC is one of these processes that allows the design, testing and implementation of secure software. It has received quite broad attention and many vendors use it to guideline their software development. The SDLC divides the software development into 6 steps:

1. Requirements and use cases
2. Design
3. Test plans
4. Code
5. Test results
6. Field Feedback

As this framework describes a live cycle, this process needs to be repeated when finished to implement the gathered feedback. The SDLC already highlights the importance of software testing although security measures can be taken in each step of the SDLC. [5]

A Systematic Mapping Study on Security Approaches in Secure Software Engineering was conducted in 2021. It revealed that many earlier conducted studies suggest to implement security measures in the design phase of the SDLC. The second most popular suggestion is to implement these measures through out the whole SDLC. Third, 23% suggest to implement security measures within the testing phase of the SDLC. [14] This clearly demonstrates the importance and popularity of implementing software security measures throughout the SDLC.

In the following, different software testing methods will be presented as the goal of this term paper is to evaluate how AI supports penetration testing. Afterwards the focus will be put to penetration testing as one of these testing techniques.

Already before a softwares ran once a Static Application Security Test (SAST) scanner is able to locate vulnerabilities in it. It can detect vulnerabilities in the source code itself as well as in byte or machine code. The weakness of these scanners is that they tend to have high false positive rates. [15]

Real attack scenarios like the exploitation of XSS and SQL injection are performed by Dynamic Application Security Test (DAST) scanners. DAST scanners have no access to the source code of an application neither to any run

time information like logs. [16] They systematically search for standard vulnerabilities like insecure headers and unsecured endpoints. Because of this rather unadapted test strategy they are not capable of finding logic errors or performing creative attacks.

4 PENETRATION TESTING

"[P][...]enetration testing is a detailed procedure to unravel [...] open doors in a system or network." [6] The technique of Penetration Testing is a creative process of understanding IT-systems and their detailed structure and functionality. Finding and documenting vulnerabilities in IT-infrastructure helps to secure them and enhance the safety of these systems.

DAST scanners are able to test software for vulnerabilities by replaying known attack patterns. In comparison to this, Penetration Tests are more creative approaches. A tester adopts his testing plans based on the reactions of the System Under Test and sets up his plans based on architectural details. Penetration Tests in fact are able to simulate full blown realistic attacks against a System Under Test. Findings from these realistically simulated attacks reveal in detail how real world attackers could exploit existing vulnerabilities. These information are being documented in a test report to inform "IT infrastructure management" [6] and propose possible mitigation strategies.

If a corporate vendor decides to conduct a Penetration Test on its system this test can be performed either by its own employees or by a third party specializes company. Either way the goal of finding exploitable vulnerabilities needs to be specified in detail. You can refer to the example of stealing a car. By a Penetration Test a specialist can find out how hard it is to steal your car but this appears not to be quite specific. Stealing a car usually refers to someone driving another's car without permission and not returning it. Does a unknown driving one's car without permission refers to stealing it when it is returned next day? Does a teenager driving its parents car without parents permission refer to stealing it? The goal of finding out how hard it is to steal a car is simply not specific enough. For IT infrastructure a specific goal could for example be: Is a unauthorized party able to interfere with Confidentiality Indtegrity Availability (CIA) triad? [17]

In the next step it is important to consider the knowledge the Penetration Tester has about the System Under Test. Potential attackers can allocate massive resources in form of time and money if the exploitation seams worthwhile. Through providing Penetration Testers with information about the System Under Test they can "compensate for the disparity in resources." [17] Also penetration testers can be given partially internal access to simulate the scenario of already overcome first barriers by attackers. [17] Penetration Testing with different degrees of knowledge can be categorized into Black-, Grey- and White-Box Penetration Testing. Black-Box Penetration Testing means that the Penetration Tester has no internal knowledge about the System Under Test. White-Box Testing means the Tester has complete knowl-

edge of the internals and Grey-Box refers to partial knowledge of internal structures. [8]

5

AI DRIVEN PENETRATION TESTING

Penetration Tests can either be performed by a software vendor it self or they can be delegated to a third party specialist. Costs that could be caused by data breaches and exploited vulnerabilities are significantly higher than costs of Penetration Tests. [18] However Penetration Tests are time consuming and labour intensive thus expensive dough. [10]

Tools like BurpSuite, ZAP and SQLMap already implement automated Penetration Testing techniques. They can be compared to DAST scanning approaches. These automated scanners can support and enhance the process of Penetration Testing but these approaches do not show enough creativity to compete with human testers. [9].

Earlier approaches to automate the process of Penetration Testing leveraged Reinforcement Learning. These AI models were given a skill set e.g. networking tools like ping and nmap as well as concrete exploits. In training landscapes they were then trained to use these tools beneficially. [10] Scaling these Reinforcement Learning approaches to an production ready state brought up difficulties. One limiting factor is the need of high fidelity training environments. Also the maximum network size in which Penetration Tests can be performed by these tools is limited and not suitable for corporate networks. As the AI needs to be provided and trained with individual exploits the set of these exploits is limited in size and adoption speed. [11]

Leveraging LLMs for penetration testing shows better results and scalability in comparison to Reinforcement Learning. GPT-3.5 and GPT-4 without any technical additions are already capable of solving Capture The Flag (CTF) challenges. They are prompted to advice the user to execute a specific task like running a Command Line Interface (CLI) command which's output the user prompts back to the LLM. This proves that LLMs are already capable of performing Penetration Tests based on their built in domain knowledge. [12] The whole Penetration Testing process can be split up into multiple tasks. The architecture of PentestGPT introduced three agents to split these tasks. One agent responsible for information condensing, one for reasoning and one for concrete task planing. This agentic approach further improves the CTF challenge completion rate. [12]

Now that LLMs are capable of performing Penetration Tests in general a limiting factor is the seek of domain specific knowledge regarding specific known exploits. The project PentestAgent introduces this domain specific knowledge. It equips the agentic setup with a Retrieval-augmented generation (RAG). Through this external knowledge base multiple vulnerability databases are introduces. Through several adjustments regarding the agentic setup and the additionally introduces domain specific knowledge the performance of LLMs in Penetration Testing was improved even further. [19]

The success of PentestGPT and PentestAgent are not limited to small networks or even single hosts. AI Agents and thus LLMs are also capable of performing Penetration Tests in real world and real size networks. ARTEMIS is a agentic approach like PentestGPT and PentestAgent. However it was optimized and tested in a real world university network. Just with the domain knowledge of GPT-5 it was able to compete with human penetration testers in terms of severity and complexity of findings. As mentioned by the architects of ARTEMIS this application tends to produce false positives. It scans systems in a broad mana instead of moving lateral like experience human penetration testers do. [20]

The vulnerabilities that PentestGPT, PentestAgent and ARTEMIS exploited in the benchmarks were nearly all performed based on known vulnerabilities or in controlled and thus not realistic test environments. For cooperate vendors especially web application Penetration Testing is highly interesting. This is because web applications are highly exposed and popular. Especially ARTEMIS and PentestAgent aims different fields than that. They concentrate more on network based attack and exploitation of known vulnerabilities. For finding new vulnerabilities in unknown web application environments these tools are not made and tested for.

Never the less these studies found that LLMs are already capable of performing specific typed of Penetration Tests with their inherit domain knowledge. The usage of RAG improves the Penetration Testing capability even further. These promising developments in using LLMs for Penetration Testing are adopted by industrial software vendors like Google. Google published oss-fuzz, a LLM powered fuzzing tool that actively found many security flaws in open source software. [21] An other example is XBOW which offers LLM based Penetration Tests in real world cooperate environments. [22]

Real world applications are much more complicated and unknown than test environments like DVWA, OWASP Juice Shop and OWASP WebGoat. They have dynamic pages with JavaScript logic and proprietary, sometimes niche technology stacks. Automated testing tools need to traverse the detailed architecture of the system under test. XBOW's high position on the Hackerone leaderboard proves that they are clearly capable of making valid findings in corporate infrastructure. [23] [24]

Corporate security experts who used XBOW on proprietary web applications shared their experiences for this term paper. According to the experts, XBOW is indeed capable of finding vulnerabilities, but they report that the tool tends to produce a high rate of false positives. The experts also note that valid findings made by XBOW are generally low-hanging fruits from the perspective of experienced penetration testers.

When it comes to proprietary technology stacks of web applications, the experts state that XBOW is not able to dig deep enough into the inner workings of proprietary frameworks. In their tests, the researchers used a deliberately vulnerable web application based on a proprietary technology stack as a benchmark system. The experts found that XBOW was unable to identify any vulnerabilities in this system.

The security expert who conducted these tests concludes that XBOW lacks the capability for lateral movement required to thoroughly investigate the working principles of the application.

6 CONCLUSION AND OUTLOOK

6.1 THIS IS POSSIBLE

Before capable LLMs were widely available and common, AI Models trained through Reinforcement Learning were leveraged for Penetration Testing. Several Penetration Testing and Security Tools implemented these technique to support human testers. However to compete with human Penetration Testers these tools lack creativity and flexibility to adopt to real world Systems Under Test. For example, AI models trained with Reinforcement Learning are useful when it comes to classification of a webapp's input fields. This classification can then be used to insert specific test requests for specific fields. This however is still quite static and doesn't address specifics of a given webapp.

With the rise of LLMs into the public more and more research was conducted on leveraging LLMs for automated Penetration Testing. Research revealed that already the inherit knowledge of a LLM is capable of performing Penetration Tests. Penetration Testing with these is as simple as asking a publicly available LLM like Mistral, Kimi-k or GPT to assist. The user then has to execute what the LLM suggests to do. For example executing a CLI command or opening a browser window.

By providing the LLM with tool usage capabilities this process can be automated further. Also the inherit knowledge of the LLM can be extended with external sources like Vulnerability Databases. With this it is possible to build fully automated Penetration Testing agents that actually find vulnerabilities.

6.2 FUNCTIONAL LIMITATIONS

LLMs are capable of performing Penetration. However, these tend to show high false positive rates which can only partially be mitigated. Especially noteworthy is that they are often benchmarked with popular deliberately vulnerable applications like DVWA, OWASP Juice Shop and OWASP WebGoat. The vulnerabilities in these applications are widely known and there fore well represented in LLMs training data. It may therefore be that LLMs perform well at finding vulnerabilities in these webapps.

When it comes to performing Penetration Tests in proprietary tech stacks or applications built with niche frameworks, automated Penetration Tests leveraging LLMs reach their limits. Tests with proprietary tools like XBOW revealed that even highly sophisticated tools struggle with niche and proprietary tech stacks. In general they are capable of finding issues in real world bug bounty programs though, as tools like XBOW are placed at the top of Hackerone leaderboard. [23] [24]

6.3 SECURITY CHALLENGES

LLMs are trained based on a set of data. These data do not encompass the entirety of available information. As not even the entirety of information necessarily represents truth, LLMs have inherit bias from their training data. This can lead to unwanted treatment of actual vulnerabilities like overlook. [25]

In the context of Penetration Testing LLMs are often times equipped with tools like web search. Prompts can be injected into the LLM's context through the access of third party webpages. In the combination with the LLM holding confidential data, the risk of leaking these information to a third party webpage rises.

6.4 FUTURE WORK

With an appropriate setup, LLMs can conduct end-to-end penetration tests. However, their performance in identifying vulnerabilities remains limited when applied to proprietary or niche technology stacks. Future research should prioritize investigating the underlying causes of these limitations and developing strategies to enhance AI-driven testing in such environments. Additionally, further study is needed to address and mitigate the security challenges associated with these approaches.

Part II

APPENDIX

BIBLIOGRAPHY

[1] T. Given-Wilson, E. Baranov, and A. Legay, "Building user trust of critical digital technologies," in *2020 IEEE International Conference on Industrial Technology (ICIT)*, Feb. 2020, pp. 1199–1204. DOI: 10.1109/ICIT45562.2020.9067154.

[2] L. A. Gordon, M. P. Loeb, and T. Sohail, "Market value of voluntary disclosures concerning information security1," *Management Information Systems Quarterly*, vol. 34, no. 3, pp. 567–594, Sep. 2010, ISSN: 0276-7783. DOI: 10.2307/25750692.

[3] European Union, *Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing directive 95/46/ec (general data protection regulation)*, Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88, 2016. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679.

[4] A. Adeyeri and H. Abroshan, "Geopolitical ramifications of cybersecurity threats: State responses and international cooperations in the digital warfare era," *Information*, vol. 15, no. 11, 2024, ISSN: 2078-2489. DOI: 10.3390/info15110682.

[5] B. Potter and G. McGraw, "Software security testing," *IEEE Security & Privacy*, vol. 2, no. 5, pp. 81–85, Sep. 2004, ISSN: 1558-4046. DOI: 10.1109/MSP.2004.84.

[6] P. Lachkov, L. Tawalbeh, and S. Bhatt, "Vulnerability assessment for applications security through penetration simulation and testing," *Journal of Web Engineering*, vol. 21, no. 7, pp. 2187–2208, Oct. 2022, ISSN: 1544-5976. DOI: 10.13052/jwe1540-9589.2178. [Online]. Available: https://ieeexplore.ieee.org/document/10251051.

[7] Y. Stefinko, A. Piskozub, and A. Obshta, "Analysis of vulnerability characteristics for automated penetration testing," in *2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, Oct. 2024, pp. 449–453. DOI: 10.1109/TCSET64720.2024.10755620.

[8] N. Rane and A. Qureshi, "Comparative analysis of automated scanning and manual penetration testing for enhanced cybersecurity," in *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, Apr. 2024, pp. 1–6. DOI: 10.1109/ISDFS60797.2024.10527240.

[9] T. A. Osman, J. R. Ragasa, H. Posesione, A. David, and M. Shakiba, "Comparative analysis of penetration testing tools in assessing web application vulnerabilities," in *2025 Interdisciplinary Conference on Electrics and Computer (INTCEC)*, 2025, pp. 1–6. DOI: 10.1109/INTCEC65580.2025.11256013.

[10] A. Confido, E. V. Ntagiou, and M. Wallum, "Reinforcing penetration testing using ai," in *2022 IEEE Aerospace Conference (AERO)*, 2022, pp. 1–15. DOI: 10.1109/AERO53065.2022.9843459.

[11] J. Schwartz and H. Kurniawati, "Autonomous penetration testing using reinforcement learning," *ArXiv*, vol. abs/1905.05965, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:155090057.

[12] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, Aug. 2024, pp. 847–864, ISBN: 978-1-939133-44-1. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng.

[13] M. Kozlovska, A. Piskozub, and V. Khoma, "Artificial intelligence in penetration testing: Leveraging ai for advanced vulnerability detection and exploitation," *ACPS*, vol. 10, no. 1, pp. 65–70, 2025. DOI: https://doi.org/10.23939/acps2025.01.065.

[14] R. A. Khan, S. U. Khan, H. U. Khan, and M. Ilyas, "Systematic mapping study on security approaches in secure software engineering," *IEEE Access*, vol. 9, pp. 19 139–19 160, 2021, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3052311.

[15] D. Dalaq, K. F. Daya, A. Dalaq, M. N. Arefin, and M. K. Niazi, "A systematic literature review on static application security testing (sast) tools: Evaluation, benchmarks, challenges, and future directions," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, ser. EASE Companion '25, New York, NY, USA: Association for Computing Machinery, 2025, 162–168, ISBN: 9798400718328. DOI: 10.1145/3727967.3756838. [Online]. Available: https://doi.org/10.1145/3727967.3756838.

[16] R. Singh, M. Kumar Gupta, D. R. Patil, and S. Maruti Patil, "Analysis of web application vulnerabilities using dynamic application security testing," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, 2024, pp. 1–6. DOI: 10.1109/I2CT61223.2024.10543484.

[17] M. Bishop, "About penetration testing," *IEEE Security & Privacy*, vol. 5, no. 6, pp. 84–87, 2007, ISSN: 1558-4046. DOI: 10.1109/MSP.2007.159.

[18] A. A. Gaafar, K. M. Fouad, and M. A. Sadek, "Penetration testing: A cost-benefit analysis of best practices implementation for software startups," in *2024 6th Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, Oct. 2024, pp. 242–245. DOI: 10.1109/NILES63360.2024.10753172.

[19] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "Pentestagent: Incorporating llm agents to automated penetration testing," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '25, New York, NY, USA: Association for Computing Machinery, 2025, pp. 375–391, ISBN: 9798400714108. DOI: 10.1145/3708821.3733882. [Online]. Available: https://doi.org/10.1145/3708821.3733882.

[20] J. W. Lin, E. K. Jones, D. J. Jasper, *et al.*, *Comparing ai agents to cybersecurity professionals in real-world penetration testing*, 2025. arXiv: 2512.09882 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2512.09882.

[21] A. Happe and J. Cito, *On the surprising efficacy of llms for penetration-testing*, 2025. arXiv: 2507.00829 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2507.00829.

[22] *Xbow: Autonomous offensive security platform*, https://xbow.com/, Accessed: 2026-02-20, XBOW, 2026.

[23] HackerOne, *Leaderboard: All time reputation (q1 2026, us, web app, owasp a1)*, Accessed: 2026-02-23, 2026. [Online]. Available: https://hackerone.com/leaderboard/all_time_reputation?year=2026&quarter=1&owasp=a1&country=US&assetType=WEB_APP&tab=bbp&userTypeTab=collective.

[24] Arch1t3ct, *Behind the hype: Is xbow ai really the "game-changer" they claim it to be?* Accessed: 2026-03-06, 2025. [Online]. Available: https://godaccess.substack.com/p/behind-the-hype-is-xbow-ai-really-the-game-changer.

[25] E. Hilario, S. Azam, J. Sundaram, K. Imran Mohammed, and B. Shanmugam, "Generative ai for pentesting: The good, the bad, the ugly," *International Journal of Information Security*, vol. 23, no. 3, pp. 2075–2097, 2024, ISSN: 1615-5270. DOI: 10.1007/s10207-024-00835-x. [Online]. Available: https://doi.org/10.1007/s10207-024-00835-x.